Taylor Jordan

Prof. Geetika Vyas

IT 6203 – IT Design Studio

October 22, 2026

Milestone 1: Landlord Profile Management

This milestone adds the Landlord Profile component to Property Pulse (Group 8). It is built around a Reactive Form that creates and edits landlord profiles (name, email, phone and properties owned) with validation, and a landlord list that shows each profile's reputation score and supports editing and archiving (soft delete). Data is stored in a TypeScript array, since a database is not required for Milestone 1.

Team member: Taylor Jordan

Assigned component: Landlord Profile Management

Repository folder: `taylor-jordan-landlord-profiles`

GitHub repository: https://github.com/jdixo137/Property-Pulse

Section 1: Labeled Code Snippets

TypeScript (landlord-profile.ts)

Snippet 1: Imports and Component Configuration

`ReactiveFormsModule` provides the `formGroup` and `formControlName` directives, and `NgClass` provides the `[ngClass]` directive. Because `LandlordProfile` is a standalone component, both are listed in the component's `imports` array.

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, Validators } from '@angular/forms';
import { NgClass } from '@angular/common';

@Component({
  imports: [ReactiveFormsModule, NgClass],
  selector: 'app-landlord-profile',
  styleUrl: './landlord-profile.css',
  templateUrl: './landlord-profile.html',
})
```

Snippet 2: Landlord Data Model

The `Landlord` interface defines each record using the data elements from the project proposal: Landlord ID, name, contact information (email and phone), properties owned, average rating and total reviews. `isArchived` supports the soft delete.

```
export interface Landlord {
  landlordId: number;
  landlordName: string;
  email: string;
  phone: string;
```

```
  propertiesOwned: number;
  averageRating: number;
  totalReviews: number;
  isArchived: boolean;
}
```

Snippet 3: Reactive Form Setup

`landlordForm` is a `FormGroup` with four `FormControls`. Each control has validators: the name is required with a minimum of 2 characters, the email must be valid, the phone must match the format 000-000-0000, and properties owned cannot be negative. `nonNullable: true` resets each field to its starting value instead of `null`.

```
landlordForm = new FormGroup({
  landlordName: new FormControl('', { nonNullable: true, validators: [Validators.required, Validators.minLength(2)] }),
  email: new FormControl('', { nonNullable: true, validators: [Validators.required, Validators.email] }),
  phone: new FormControl('', { nonNullable: true, validators: [Validators.required, Validators.pattern(/^\d{3}-\d{3}-\d{4}$/)] }),
  propertiesOwned: new FormControl(0, { nonNullable: true, validators: [Validators.required, Validators.min(0)] }),
});

get landlordName() { return this.landlordForm.controls.landlordName; }
get email() { return this.landlordForm.controls.email; }
get phone() { return this.landlordForm.controls.phone; }
get propertiesOwned() { return this.landlordForm.controls.propertiesOwned; }
```

Snippet 4: Create and Update (onSubmit)

One method handles both Create and Update. When `editingId` is `null`, a new landlord is added to the array with a new ID and no reviews yet (Create). Otherwise, the selected landlord is found by ID and its values are replaced with the form values (Update). `getRawValue()` returns the form's values as one object.

```
onSubmit() {
  const formValue = this.landlordForm.getRawValue();
  if (this.editingId === null) {
    this.landlords.push({
      landlordId: this.nextId++,
      ...formValue,
      averageRating: 0,
      totalReviews: 0,
      isArchived: false
    });
    this.message = formValue.landlordName + ' was added.';
  } else {
    const landlord = this.landlords.find(l => l.landlordId === this.editingId);
    if (landlord) {
      Object.assign(landlord, formValue);
    }
    this.message = formValue.landlordName + ' was updated.';
  }
  this.editingId = null;
  this.landlordForm.reset();
}
```

Snippet 5: Edit, Archive and Restore

`editLandlord()` uses `setValue()` to load a landlord into the form for editing. Delete is implemented as a soft delete: `archiveLandlord()` sets `isArchived` to `true`, which hides the profile instead of removing it, and

restoreLandlord() reverses it. visibleLandlords returns only active landlords unless Show Archived is turned on.

```
editLandlord(landlord: Landlord) {
  this.editingId = landlord.landlordId;
  this.landlordForm.setValue({
    landlordName: landlord.landlordName,
    email: landlord.email,
    phone: landlord.phone,
    propertiesOwned: landlord.propertiesOwned
  });
}

archiveLandlord(landlord: Landlord) {
  landlord.isArchived = true;
  this.message = landlord.landlordName + ' was archived.';
}

restoreLandlord(landlord: Landlord) {
  landlord.isArchived = false;
  this.message = landlord.landlordName + ' was restored.';
}

get visibleLandlords(): Landlord[] {
  return this.landlords.filter(landlord => this.showArchived || !landlord.isArchived);
}
```

HTML Template (landlord-profile.html)

Snippet 6: Form Connected to the FormGroup

[formGroup]="landlordForm" binds the form to the FormGroup in TypeScript, and formControlName connects each input to its FormControl. (ngSubmit) is event binding that calls onSubmit(). Each @if block shows a validation message only after the user has left an invalid field (touched). The heading uses interpolation to switch between Create and Edit mode.

```
<h2>{{ editingId === null ? 'Create Landlord Profile' : 'Edit Landlord Profile' }}</h2>
<form [formGroup]="landlordForm" (ngSubmit)="onSubmit()">
  <div class="form-field">
    <label for="landlordName">Landlord Name</label>
    <input type="text" id="landlordName" formControlName="landlordName">
    @if (landlordName.invalid && landlordName.touched) {<p class="error"> landlord name is required. Please type
at least 2 Characters. </p>}

    <label for="email">Email</label>
    <input type="text" id="email" formControlName="email">
    @if (email.invalid && email.touched) {<p class="error">Please enter a valid email address.</p>}

    <label for="phone">Phone</label>
    <input type="tel" id="phone" formControlName="phone" placeholder="000-000-0000">
    @if (phone.invalid && phone.touched) { <p class="error">Use the format 000-000-0000.</p> }

    <label for="propertiesOwned">Properties</label>
    <input id="propertiesOwned" type="number" min="0" formControlName="propertiesOwned">
    @if (propertiesOwned.invalid && propertiesOwned.touched) { <p class="error"> How Many Properties do you
```

```
manage?</p>}
  </div>
```

Snippet 7: Submit and Cancel Buttons

[disabled]="landlordForm.invalid" is property binding that keeps the submit button disabled until every field is valid. Interpolation switches the button label between Create Landlord and Save Changes, and the Cancel button appears only in Edit mode.

```
<div class="form-actions">
  <button type="submit" class="btn btn-primary" [disabled]="landlordForm.invalid">{{ editingId === null ? 'Create
Landlord' : 'Save Changes' }}</button>
  @if (editingId !== null) {
    <button type="button" class="btn btn-secondary" (click)="cancelEdit()">Cancel</button>
  }
</div>
</form>
@if (message) {<p class="success">{{ message }}</p>}
```

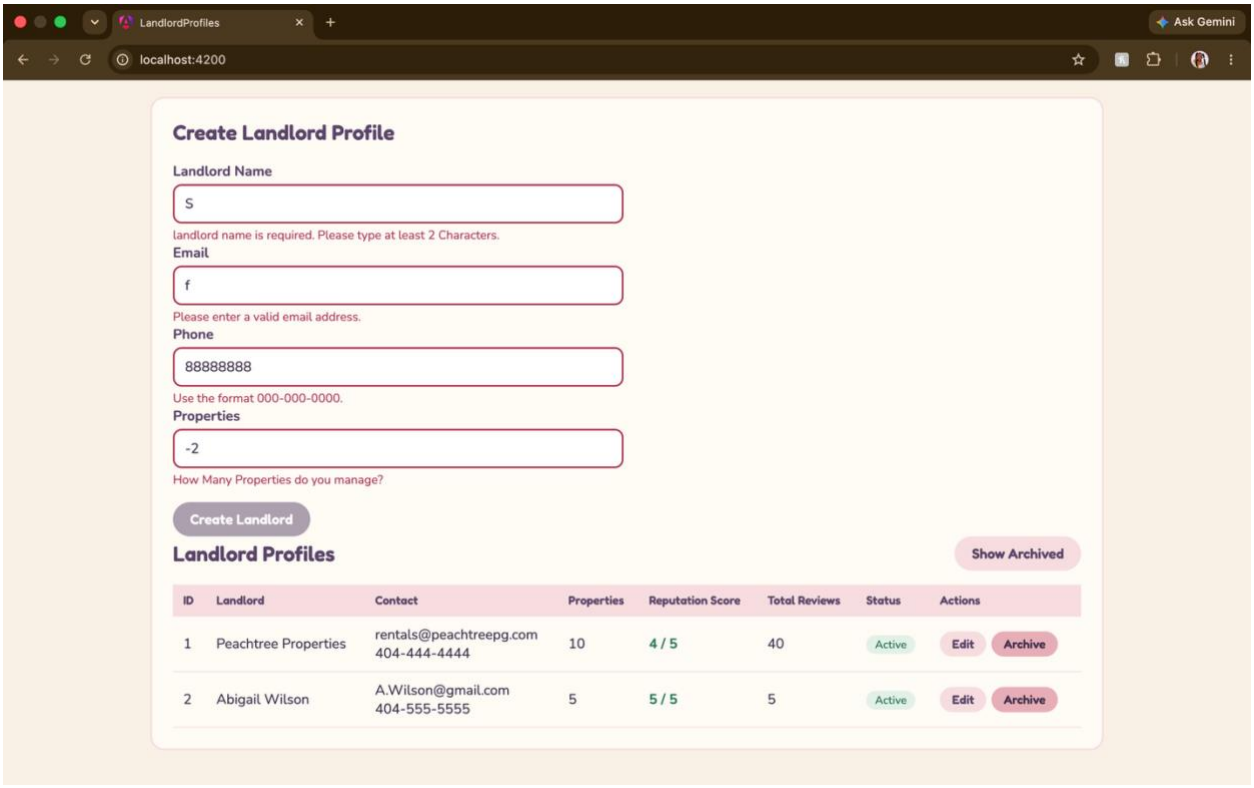
Snippet 8: Landlord List with Directives (Read)

@for repeats one table row for each visible landlord (Read). [ngClass] adds CSS classes from the data: faded rows for archived landlords, a highlight for the row being edited, colored reputation scores and status badges. @if / @else shows Edit and Archive for active landlords, and Restore for archived ones. (click) event binding passes the selected landlord to each method.

```
@for (landlord of visibleLandlords; track landlord.landlordId) {
  <tr [ngClass]="{ 'archived-row': landlord.isArchived, 'editing-row': landlord.landlordId === editingId }">
    <td>{{ landlord.landlordId }}</td>
    <td>{{ landlord.landlordName }}</td>
    <td>{{ landlord.email }}<br>{{ landlord.phone }}</td>
    <td>{{ landlord.propertiesOwned }}</td>
    <td>
      <span class="rating" [ngClass]="getRatingClass(landlord)">{{ landlord.totalReviews > 0 ?
landlord.averageRating + ' / 5' : 'No reviews yet' }}</span>
    </td>
    <td>{{ landlord.totalReviews }}</td>
    <td>
      <span class="status-badge" [ngClass]="landlord.isArchived ? 'status-archived' : 'status-active'">{{
landlord.isArchived ? 'Archived' : 'Active' }}</span>
    </td>
    <td class="actions">
      @if (!landlord.isArchived) {
        <button type="button" class="btn btn-small" (click)="editLandlord(landlord)">Edit</button>
        <button type="button" class="btn btn-small btn-danger"
(click)="archiveLandlord(landlord)">Archive</button>
      } @else {
        <button type="button" class="btn btn-small" (click)="restoreLandlord(landlord)">Restore</button>
      }
    </td>
  </tr>
} @empty {
  <tr><td colspan="8" class="empty">No landlord profiles to show.</td></tr>
}
```

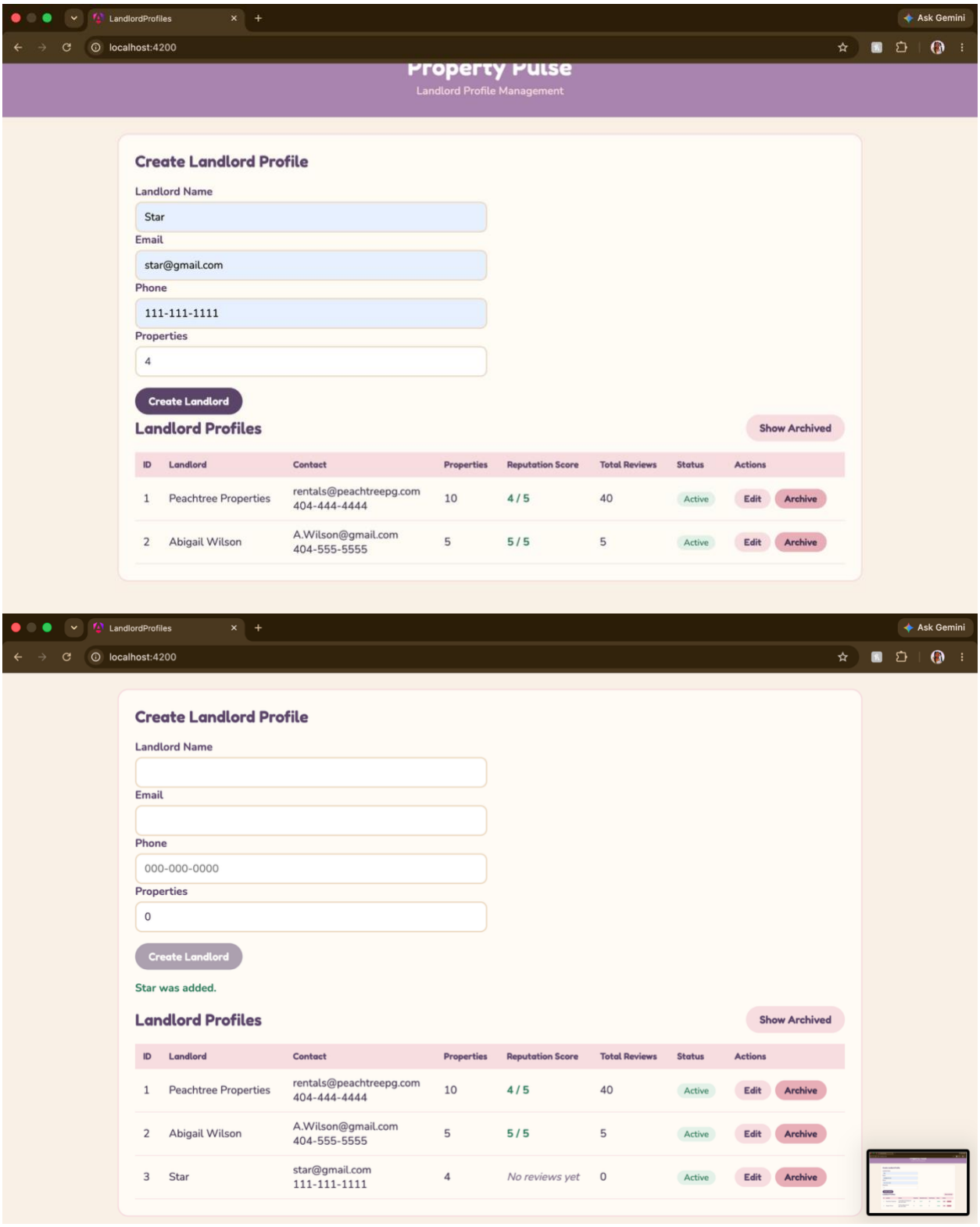
Section 2: Labeled Screenshots

Screenshot 1: Form Validation



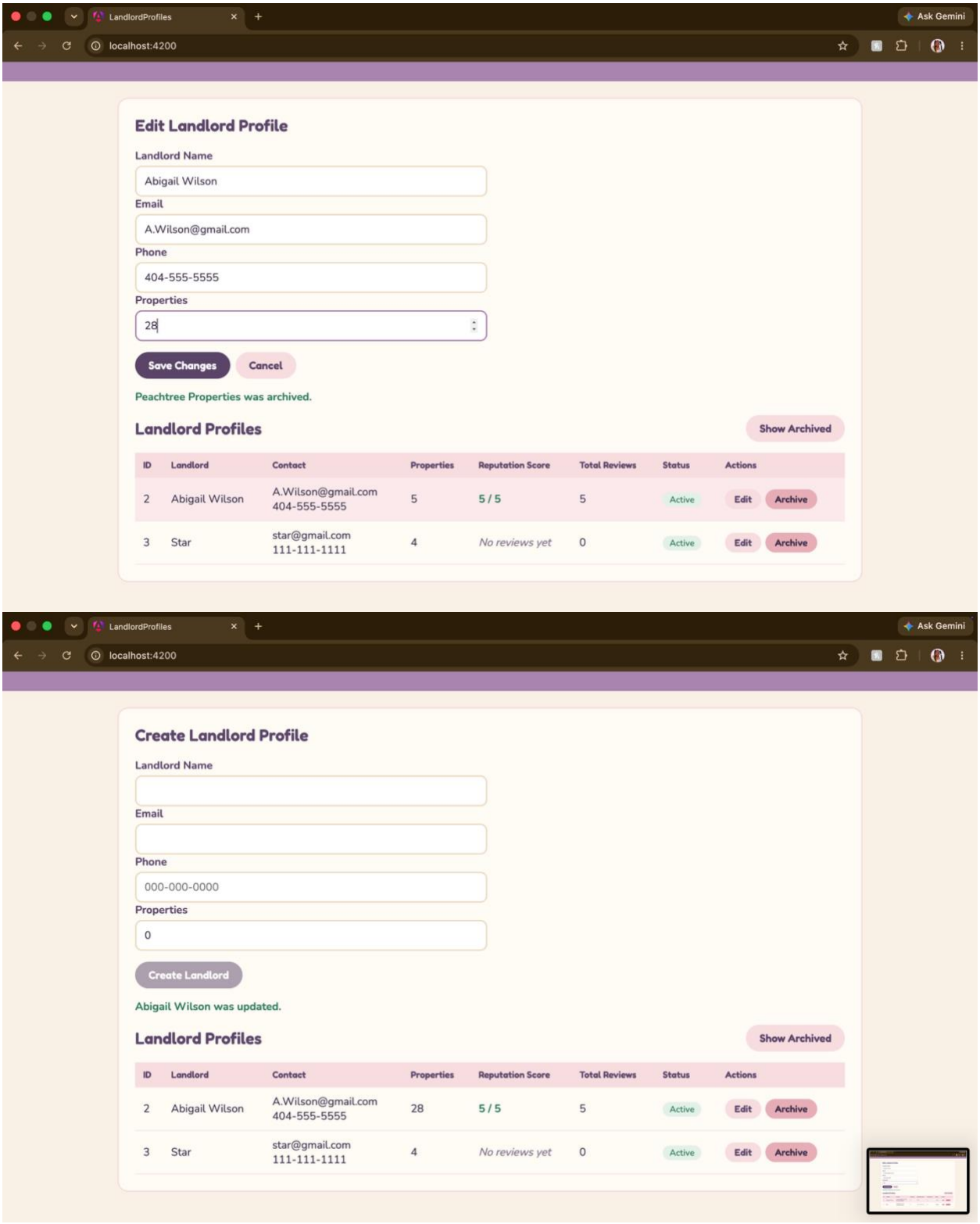
Invalid fields show a red border and the matching error message from their validators, and the Create Landlord button stays disabled until the form is valid.

Screenshot 2: Create a Landlord Profile



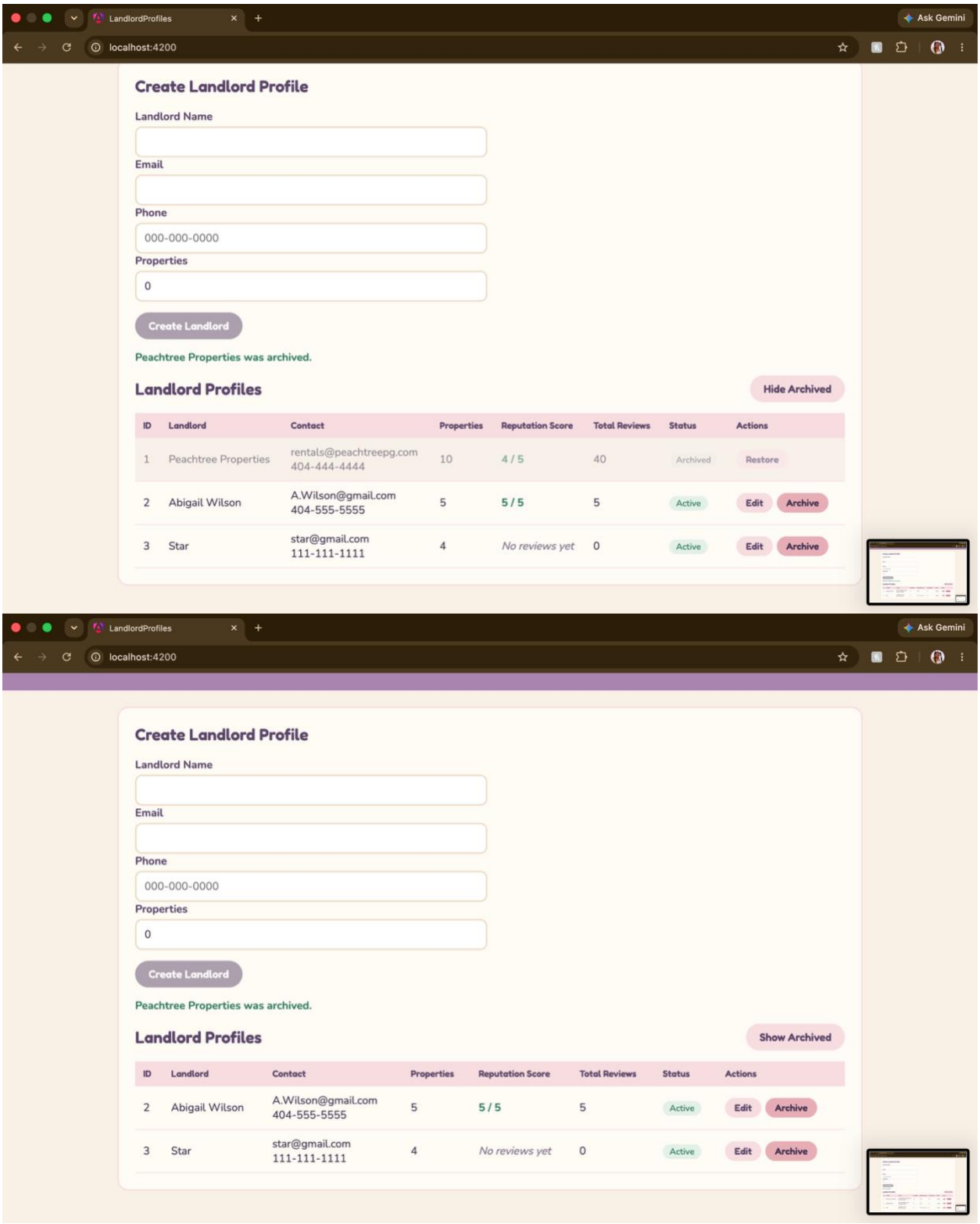
A new landlord submitted through the reactive form appears in the list with ID 3, “No reviews yet” and an Active status, and a confirmation message shows below the form.

Screenshot 3: Update a Landlord Profile



Clicking Edit loads the landlord into the form with `setValue()`. The heading changes to Edit Landlord Profile, the button changes to Save Changes, a Cancel button appears, and the row being edited is highlighted.

Screenshot 4: Archive a Landlord Profile (Soft Delete)



The archived landlord is hidden from the list by default. With Show Archived turned on, it appears faded with an Archived badge and a Restore button, and a confirmation message shows below the form.

Section 3: Shared and Component Styles

Styles the whole group shares, such as colors, fonts, cards, form fields, buttons and tables, are in src/styles.css. Colors are defined once as CSS variables so every component can use the same theme. Styles used only by this component, such as archived and edited row colors, reputation score colors and status badges, stay in landlord-profile.css.

Snippet 9: Shared Theme Variables (src/styles.css) – The theme I'm using is a placeholder.

```
:root {
  --color-cream: #f8f0e6;
  --color-blush: #f7dde0;
  --color-rose: #e8aeb7;
  --color-mauve: #a984ae;
  --color-plum: #5a4468;
  --color-text: #3d2e47;
}

h1, h2, .btn, th {
  font-family: 'Fredoka', 'Nunito', sans-serif;
}

.btn-primary { background: var(--color-plum); color: #ffffff; }
.btn-secondary { background: var(--color-blush); color: var(--color-plum); }
```

Snippet 10: Component-Only Styles (landlord-profile.css)

```
.archived-row { opacity: 0.6; background: #f3ebe1; }
.editing-row { background: #fbe9ec; }
.rating-high { color: #2f7a5a; }
.rating-low { color: #b23a48; }
.status-active { background: #e3f1e8; color: #2f7a5a; }
.status-archived { background: #ece8e0; color: #6b6475; }
```

Reference

Anthropic. (2026). *Claude* (Opus 5.5) [Large language model]. https://claude.ai

AI Use Statement

I used Claude (Anthropic) as an assistant while completing this milestone. It provided a step-by-step walkthrough for building the Landlord Profile component, including the reactive form, validators, CRUD methods and the @if, @for and [ngClass] directives. It helped me troubleshoot errors as I built the component. Claude drafted the structure of this report. I wrote, ran and tested all code myself in VS Code and verified the results in the browser.